Northeastern University

EECE 7205
Fundamentals of Computer Engineering

Mini Project: Airport Code Lookup Service
with a Binary Search Tree

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201
Catalina Buitano
NUID: 002330106

November 2026

Contents

1 Introduction **3**

1.1 Code structure 3

1.2 Recursive and iterative versions 3

2 Task 1: Loading the Data and Building the Tree **4**

2.1 Pseudocode 4

2.2 Results 6

3 Task 2: Search **7**

3.1 Pseudocode 7

3.2 Test cases 8

4 Task 3: Delete **9**

4.1 Pseudocode 9

4.2 Test cases 10

4.3 Adding a made-up airport (Task 3c) 11

5 Task 4: Traversals and Validation **12**

5.1 Pseudocode 12

5.2 Results 13

6 Task 5: Range Queries **14**

7 Task 6: The Shape Experiment **15**

7.1 Why the file order is so much worse than a shuffle 16

7.2 Connection to $O(h)$ 16

8 Recursive vs. Iterative Versions **17**

9 Interactive Session **18**

10 Checklist Against the Handout **19**

11 Problems We Ran Into **20**

12 Conclusion **21**

Data Credit **21**

A How to Build and Run **21**

1 Introduction

Every flight booking, boarding pass and baggage tag identifies an airport by its three-letter IATA code (BOS for Boston Logan, JFK for New York Kennedy). In this project we built a lookup service for airports on our own binary search tree (BST), keyed by IATA code. We load the real OpenFlights file `airports.dat`, build the tree, and implement search, insert, delete, traversals, validation and range queries. In the last part we build the same tree in different insertion orders and measure how the order changes the height, and with it the $O(h)$ running time of every operation.

1.1 Code structure

All the code is in one file, `src/main.cpp`, with the two classes the handout asks for:

- **`Airport`**: the record. Its fields (ID, name, city, country, IATA, ICAO, latitude, longitude) are private and are read through getters such as `getiata()` and `getname()`.
- **`AirportBST`**: the tree. The `root` pointer and the `Node` struct are private. Most public functions are short wrappers that call a private helper with `root` (for example, `printbfs()` calls `bfshelper(root)`).

Outside the classes are the loader (`splitline`, `loadairports`) and a few helpers (`buildtree`, `loadpreorder`, `timesearches`, `printairportbycode`). `main()` runs the test cases for every task in order, then opens an interactive session (Section 9).

1.2 Recursive and iterative versions

We wrote the core operations in two ways, a recursive version and an iterative (loop) version. Table 1 lists them. The iterative versions are the ones the program uses by default, because the handout notes that inserting 6,072 sorted keys builds a chain 6,071 nodes deep, and recursion that deep can overflow the call stack. Section 8 shows that both versions give identical results.

Operation	Recursive version	Iterative version
Insert	`insertrecursive` → `insertBSTrecursive`	`insert` → `insertBSTiterative`
Search	`findrecursive` → `searchrecursivehelper`	`find` → `searchhelper`; `contains`
Minimum	`findminrecursive` → `minnoderecursive`	`findmin`
Maximum	`findmaxrecursive` → `maxnoderecursive`	`findmax`
Delete	`removerecursive` → `removerecursivehelper`	`remove`

Table 1: Operations written both ways. The default (iterative) version is in the right column.

The traversals, height, average depth, validation and range query are recursive. Each visits every node (or every node in a range) once, so recursion is the natural way to write them, and they ran correctly even on the 6,071-deep sorted chain.

2 Task 1: Loading the Data and Building the Tree

2.1 Pseudocode

Algorithm 1 `splitline`: split one line into fields (commas inside quotes do not count)

```
1:  function SPLITLINE(line)
2:      fields ← [ ];  current ← "";  inquotes ← false
3:      for i ← 0 to len(line) − 1 do
4:          if line[i] = " then
5:              inquotes ← ¬inquotes                          ▷ the quote itself is not kept
6:          else if line[i] = ',' and not inquotes then
7:              append current to fields;  current ← ""       ▷ a real separator
8:          else
9:              current ← current + line[i]
10:         end if
11:     end for
12:     append current to fields                              ▷ the last field has no comma after it
13:     return fields
14: end function
```

Algorithm 2 `loadairports` and the build in file order

```
1:  function LOADAIRPORTS(filename, airports)
2:      open filename; if it fails return false
3:      for each line in the file do
4:          f ← SPLITLINE(line)
5:          if |f| < 8 or f[4] = \N then
6:              skip the line
7:          end if
8:          append Airport(f[0..7]) to airports   ▷ ID with stoi, latitude and longitude with stod
9:      end for
10:     return true
11: end function

12: inserted ← 0;  duplicates ← 0;  alwaysvalid ← true
13: for i ← 0 to n − 1 do
14:     if INSERT(airports[i]) then
15:         inserted ← inserted + 1
16:     else
17:         duplicates ← duplicates + 1; report airports[i].iata                    ▷ rule I4
18:     end if
19:     if not VALIDATE( ) then
20:         alwaysvalid ← false                                                     ▷ Task 4c
21:     end if
22: end for
```

Algorithm 3 Insert, iterative (`insertBSTiterative`, the default)

```
 1: function insertBSTiterative(root, airport, inserted)
 2:     newnode ← new Node(airport);  inserted ← true
 3:     if root = NULL then return newnode
 4:     end if
 5:     curr ← root
 6:     while true do
 7:         if airport.iata < curr.iata then
 8:             if curr.left = NULL then
 9:                 curr.left ← newnode; return root
10:             end if
11:             curr ← curr.left
12:         else if airport.iata > curr.iata then
13:             if curr.right = NULL then
14:                 curr.right ← newnode; return root
15:             end if
16:             curr ← curr.right
17:         else                                          ▷ duplicate (rule I4)
18:             inserted ← false; delete newnode; return root
19:         end if
20:     end while
21: end function
```

Algorithm 4 Insert, recursive (`insertBSTrecursive`)

```
 1: function insertBSTrecursive(node, airport, inserted)
 2:     if node = NULL then
 3:         inserted ← true; return new Node(airport)
 4:     end if
 5:     if airport.iata < node.iata then
 6:         node.left ← insertBSTrecursive(node.left, airport, inserted)
 7:     else if airport.iata > node.iata then
 8:         node.right ← insertBSTrecursive(node.right, airport, inserted)
 9:     else
10:         inserted ← false                              ▷ duplicate (rule I4)
11:     end if
12:     return node
13: end function
```

Algorithm 5 Height and average depth (height counts edges: one node has height 0)

```
1: function COMPUTEHEIGHT(node)
2:     if node = NULL then return −1
3:     end if
4:     return 1 + max(COMPUTEHEIGHT(node.left), COMPUTEHEIGHT(node.right))
5: end function
6: function SUMANDCOUNT(node, depth, sum, count)
7:     if node = NULL then return
8:     end if
9:     sum ← sum + depth;  count ← count + 1
10:    SUMANDCOUNT(node.left, depth + 1, sum, count)
11:    SUMANDCOUNT(node.right, depth + 1, sum, count)
12: end function
13: function AVERAGEDEPTH(root)
14:    sum ← 0;  count ← 0;  SUMANDCOUNT(root, 0, sum, count)
15:    return (double) sum/count
16: end function
```

2.2 Results

```
lines read: 7698
airports loaded: 6072
lines skipped: 1626
Inserted 6072 airports, 0 duplicates (expected 0)

Task 1c
BST size: 6072
BST height: 177
Average depth: 18.3233
```

Check	Handout	Our result
Lines in the file	7,698	7,698
Airports with an IATA code	6,072	6,072
Lines skipped (IATA is \N)	1,626	1,626
Duplicate codes (rule I4)	0	0
Height, file order	177	177
Average depth	18.3	18.3233

Table 2: Task 1 results against the facts in the handout.

3 Task 2: Search

3.1 Pseudocode

Algorithm 6 Search with path, iterative (`searchhelper`, used by `find`)

```
1: function SEARCHHELPER(code, path, comparisons)
2:     curr ← root;  path ← [ ];  comparisons ← 0
3:     while curr ≠ NULL do
4:         append curr.iata to path;  comparisons ← comparisons + 1
5:         if code = curr.iata then return curr
6:         else if code < curr.iata then
7:             curr ← curr.left
8:         else
9:             curr ← curr.right
10:        end if
11:    end while
12:    return NULL
13: end function
```

Algorithm 7 Search with path, recursive (`searchrecursivehelper`, used by `findrecursive`)

```
1: function SEARCHRECURSIVEHELPER(node, code, path, comparisons)
2:     if node = NULL then return NULL
3:     end if
4:     append node.iata to path;  comparisons ← comparisons + 1
5:     if code = node.iata then return node
6:     end if
7:     if code < node.iata then return SEARCHRECURSIVEHELPER(node.left, ...)
8:     else return SEARCHRECURSIVEHELPER(node.right, ...)
9:     end if
10: end function
```

Algorithm 8 Minimum and maximum, both ways (maximum is the same with *right*)

```
1: function FINDMIN( )                                          ▷ iterative
2:     curr ← root
3:     while curr.left ≠ NULL do
4:         curr ← curr.left
5:     end while
6:     return curr.iata
7: end function
8: function MINNODERECURSIVE(node)                              ▷ recursive
9:     if node.left = NULL then return node
10:    end if
11:    return MINNODERECURSIVE(node.left)
12: end function
```

`find(code)` calls `searchhelper` and prints the path, the number of comparisons and the airport, or "not found". `contains(code)` is the same loop without printing, used for timing in Task 6. To find a code deep in the tree, `deepestiata()` returns the last code in level order, which is always on the deepest level.

3.2 Test cases

Case	Code	Comparisons	Result
The root	GKA	1	found (Goroka Airport)
A leaf	AAC	5	found (El Arish International Airport)
Deep in the tree (deepest node)	YZR	178	found (Chris Hadfield Airport, Sarnia)
Does not exist	XYZ	14	not found
Smaller than every key	000	4	not found (falls off the left of AAA)
Larger than every key	ZZZZ	37	not found (falls off the right of ZZV)
Smallest real code	AAA	4	found (Anaa Airport)
Sample	BOS	12	found (Logan International)

Table 3: Search test cases on the file-order tree.

Minimum code: AAA. **Maximum code:** ZZV.

The code `000` is used for "smaller than every key" because digits sort before letters, so it is smaller than AAA, the smallest real code. Searching for `AAA` itself finds it, so it cannot test this case.

```
Path: GKA -> AEY -> EGS -> BUJ -> BJA -> BSK -> BOY -> BNI -> BOH -> BOO -> BOU -> BOS (
    comparisons: 12)
BOS General Edward Lawrence Logan International Airport, Boston, United States
Path: GKA -> MAG -> POM -> WWK -> YAM -> XFW -> XGN -> XLS -> XRY -> XVS -> YAI -> YAA -> XYA ->
    XYE -> (NULL) not found (comparisons: 14)
Path: GKA -> AEY -> AAE -> AAA -> (NULL) not found (comparisons: 4)
```

The path for YZR has 178 nodes (height $177 + 1$), and after the first four steps it is almost entirely Canadian Y codes in increasing order (YAM, YAY, YAZ, YBB, ...). We come back to this in Task 6.

4 Task 3: Delete

4.1 Pseudocode

Algorithm 9 Delete, iterative (`remove`, the default)

```
 1: function REMOVE(code)
 2:     parent ← NULL;  curr ← root
 3:     while curr ≠ NULL and curr.iata ≠ code do          ▷ find the node and its parent
 4:         parent ← curr
 5:         if code < curr.iata then
 6:             curr ← curr.left
 7:         else curr ← curr.right
 8:         end if
 9:     end while
10:     if curr = NULL then return false                  ▷ not found
11:     end if
12:     if curr has no children then                      ▷ D1: leaf
13:         replace curr in parent with NULL
14:     else if curr has exactly one child c then         ▷ D2
15:         replace curr in parent with c
16:     else                                              ▷ D3: two children
17:         succ ← smallest node in curr.right;  succparent ← its parent
18:         curr.data ← succ.data
19:         link succparent to succ.right (in place of succ); delete succ
20:     end if
21:     print the case and VALIDATE( )
22:     return true
23: end function
```

"Replace in parent" means: if *parent* is NULL the node was the root, so `root` changes; otherwise whichever pointer of *parent* pointed to *curr* changes.

Algorithm 10 Delete, recursive (`removerecursivehelper`, used by `removerecursive`)

```
 1: function REMOVERECURSIVEHELPER(node, code)        ▷ returns the new root of this subtree
 2:     if node = NULL then return NULL                                          ▷ not found
 3:     end if
 4:     if code < node.iata then
 5:         node.left ← REMOVERECURSIVEHELPER(node.left, code); return node
 6:     else if code > node.iata then
 7:         node.right ← REMOVERECURSIVEHELPER(node.right, code); return node
 8:     end if
 9:     if node has no children then
10:         delete node; return NULL                                              ▷ D1
11:     end if
12:     if node.right = NULL then
13:         c ← node.left; delete node; return c                          ▷ D2, left child
14:     end if
15:     if node.left = NULL then
16:         c ← node.right; delete node; return c                        ▷ D2, right child
17:     end if
18:     succ ← MINNODERECURSIVE(node.right)                                       ▷ D3
19:     node.data ← succ.data
20:     node.right ← REMOVERECURSIVEHELPER(node.right, succ.iata)     ▷ successor has no left
    child
21:     return node
22: end function
```

The recursive version needs no *parent* pointer: each call returns the new root of its subtree, and the caller stores it in its own *left* or *right* pointer.

4.2 Test cases

We found one code for each case by printing the tree, and ran each delete on a freshly built file-order tree. `printlevels(3)` prints the top three levels before and after each delete, and `remove` runs `validate` after every delete.

Case	Code	Program output	Validate
D1: leaf	AAC	case D1: leaf	OK
D2: only a left child	GJT	case D2: only a left child, GJR	OK
D2: only a right child	AAA	case D2: only a right child, AAC	OK
D3: two children	AEY	case D3: replaced by successor AFA	OK
The root (D3)	GKA	case D3: replaced by successor GKE	OK
Sample	BOS	case D2: only a left child, BOR	OK

Table 4: Delete test cases.

For AAC, GJT and AAA the top three levels do not change, because those nodes are deeper in the tree. The two deletes that do change the top levels:

remove AEY (two children)

before

```
GKA
AEY MAG
AAE EGS HGU POM
```

after

```
GKA
AFA MAG
AAE EGS HGU POM
```

remove GKA (the root)

before

```
GKA
AEY MAG
AAE EGS HGU POM
```

after

```
GKE
AEY MAG
AAE EGS HGU POM
```

4.3 Adding a made-up airport (Task 3c)

```
Added QQQ validate: OK
Path: GKA -> MAG -> POM -> WWK -> UAK -> ... -> QPW -> QQT -> QQQ (comparisons: 22)
QQQ Cats Buitano Memorial Airfield, Boston, United States
Deleted QQQ (case D1: leaf) validate: OK
Path: GKA -> MAG -> POM -> WWK -> UAK -> ... -> QPW -> QQT -> (NULL) not found (comparisons: 21)
```

QQQ goes in as a new leaf under QQT, so removing it is case D1, and the search afterwards stops at QQT with one comparison fewer.

5 Task 4: Traversals and Validation

5.1 Pseudocode

Algorithm 11 `validRange`: the (lo, hi) range method

```
1: function validRange(node, low, high)                ▷ NULL bound means "no limit"
2:     if node = NULL then return true
3:     end if
4:     if low ≠ NULL and node.iata ≤ low.iata then return false
5:     end if
6:     if high ≠ NULL and node.iata ≥ high.iata then return false
7:     end if
8:     return validRange(node.left, low, node) and validRange(node.right, node, high)
9: end function
```

Going left, the current node becomes the new upper bound (everything there must be smaller); going right, it becomes the new lower bound. The public function `isvalidBSTrangedsolution()` calls `validRange(root, NULL, NULL)`.

Algorithm 12 Inorder sorted check (`checkinorder`, Task 4a)

```
1: function checkInOrder(node, prev)                   ▷ left, node, right
2:     if node = NULL then return true
3:     end if
4:     if not checkInOrder(node.left, prev) then return false
5:     end if
6:     if prev ≠ NULL and prev.iata ≥ node.iata then return false
7:     end if
8:     prev ← node
9:     return checkInOrder(node.right, prev)
10: end function
```

Algorithm 13 Level order with a queue (`bfshelper`)

```
1: function bfsHelper(root)
2:     res ← [ ]; if root = NULL return res
3:     Q ← queue containing root
4:     while Q not empty do
5:         curr ← dequeue(Q); append curr.iata to res
6:         if curr.left ≠ NULL then enqueue(Q, curr.left)
7:         end if
8:         if curr.right ≠ NULL then enqueue(Q, curr.right)
9:         end if
10:    end while
11:    return res
12: end function
```

Algorithm 14 Save and load in preorder (Task 4b)

```
1: function SAVEPREORDER(file)
2:     write every code of PREORDER(root) to file, one per line        ▷ node, left, right
3: end function
4: function LOADPREORDER(tree, file, sorted)
5:     CLEAR(tree)
6:     for each code in file do
7:         binary-search sorted for the record with this code
8:         INSERT(tree, record)
9:     end for
10: end function
```

5.2 Results

- **4a:** `list` prints all 6,072 airports in code order, from `AAA | Anaa Airport` to `ZZV | Zanesville Municipal Airport`, and the check prints `Inorder is sorted: true`. The level order starts `GKA AEY MAG AAE EGS HGU POM` and contains all 6,072 codes.
- **4b:** `savepreorder` writes the codes to `preorder.txt`, and `loadpreorder` inserts them in that order into a new tree, using binary search on the airports sorted by code to recover each full record. The reloaded tree has height 177 (the same as the original) and `Same level order: true`. This works because in preorder every node comes before everything in its subtrees, so each node is inserted before its descendants and lands in exactly its old position.
- **4c:** `validate` (range method) was true after **every** one of the 6,072 inserts in the Task 1 build (`validate after every insert: true`), after every delete in Task 3, and after adding and removing QQQ.

6 Task 5: Range Queries

Algorithm 15 `rangehelper`: range query with pruning

```
1: function RANGEHELPER(node, from, to, res, visited)
2:     if node = NULL then return
3:     end if
4:     visited ← visited + 1
5:     if node.iata > from then RANGEHELPER(node.left, ...)
6:     end if                                  ▷ else the whole left side is below from
7:     if from ≤ node.iata ≤ to then append node.iata to res
8:     end if
9:     if node.iata < to then RANGEHELPER(node.right, ...)
10:    end if                                  ▷ else the whole right side is above to
11: end function
```

This is an inorder traversal that skips subtrees outside the range, so the results come out in alphabetical order without visiting the rest of the tree. The program prints every airport in the range (looking up each full record with `printairportbycode`), followed by the count:

```
BOS | General Edward Lawrence Logan International Airport | Boston, United States
BOU | Bourges Airport | Bourges, France
...
BWI | Baltimore/Washington International Thurgood Marshall Airport | Baltimore, United States
range BOS BWI: 113 airports (expected 113), 127 nodes visited
```

The query returns the 113 airports the handout expects and visits only 127 of the 6,072 nodes, which shows the pruning works.

7 Task 6: The Shape Experiment

Algorithm 16 Balanced build (middle rule) and `rebalance`

```
1: function BUILDBALANCEDHELPER(sorted, lo, hi)
2:     if lo > hi then return NULL
3:     end if
4:     mid ← ⌊(lo + hi)/2⌋
5:     node ← new Node(sorted[mid])                    ▷ the middle element is the root
6:     node.left ← BUILDBALANCEDHELPER(sorted, lo, mid − 1)
7:     node.right ← BUILDBALANCEDHELPER(sorted, mid + 1, hi)
8:     return node
9: end function
10: function REBALANCE( )
11:     sorted ← inorder list of the tree                ▷ already sorted by code
12:     CLEAR( );  root ← BUILDBALANCEDHELPER(sorted, 0, |sorted| − 1)
13: end function
```

We built the tree in four ways from the same 6,072 records: file order, sorted order, the balanced build, and a random shuffle (bonus). For the timing, we made one list of 10,000 random codes (`mt19937` with a fixed seed) and searched for all of them in each tree with `contains`.

Build order	Height	Avg depth	10,000 searches	Explanation
File order	177	18.32	2.98 ms	The file is mostly in ICAO order, which creates long runs of increasing IATA codes; each run becomes a near-chain to the right.
Sorted order	6,071	3,035.50	362.73 ms	Every new key is the largest so far, so it always goes right: a linked list with height $n - 1$.
Balanced (middle rule)	12	10.65	2.05 ms	The middle element is the root of each subtree, so each level halves the range: height $\lfloor \log_2 6072 \rfloor = 12$.
Random shuffle (bonus)	31	14.88	2.69 ms	A random order breaks up the runs, so no long chains form; the height is $O(\log n)$.

Table 5: Task 6 results (one run on our laptop, compiled with `-O2`).

After `rebalance()` on the file-order tree we get height 12 and average depth 10.65, the same as the balanced build. Our shuffle came out at height 31, a little above the 25 to 29 the handout gives; the height of a random tree changes from shuffle to shuffle. Its average depth, 14.88, is close to the theoretical average for a random BST, $2H_n - 4 \approx 14.6$ for $n = 6072$. The times are from a single run and change slightly each time, but the order of the rows stays the same; the heights and

average depths do not change.

7.1 Why the file order is so much worse than a shuffle

The lines of `airports.dat` are not in random order. Our program measured how ordered the file already is:

```
How the file is ordered (a random order gives about 50%):
lines with a larger IATA code than the line before: 57.7994%
lines with a larger ICAO code than the line before: 81.0904%
longest run of increasing IATA codes: 162 lines
```

So 81% of lines have a larger ICAO code than the line before, against about 50% for a random order: the file is mostly sorted by ICAO code. ICAO codes start with a regional prefix, so this also groups the airports by country. The file starts with Papua New Guinea (GKA, MAG, HGU, ...), and GKA, the first line, becomes the root.

For many countries the IATA code follows the ICAO code. Canada is the clearest case: ICAO `CYYZ` is IATA `YYZ`, ICAO `CYVR` is IATA `YVR`, and so on. When the file runs through Canada's airports in ICAO order, their IATA codes arrive almost in increasing order (our program found one unbroken increasing run of 162 lines). When keys are inserted in increasing order, each one becomes the right child of the one before it, so that run turns into a chain, a smaller version of the sorted-order tree.

That is exactly the search path for YZR, the deepest node, from Task 2:

```
GKA -> MAG -> POM -> WWK -> YAM -> YAY -> YAZ -> YBB -> YBC -> YBG -> ... -> YZH -> YZP -> YZR
```

After four steps the path is about 170 Y codes in a row, and this Canadian chain is what makes the height 177. The rest of the tree is fairly balanced, which is why the average depth (18.3) is much closer to the shuffle's than the height is. A random shuffle breaks up these runs, so the height drops to around 30.

7.2 Connection to $O(h)$

Search, insert and delete all go down one path from the root, so they cost $O(h)$. For a successful search, the number of comparisons is the node's depth plus one, so the average number of comparisons is the average depth plus one:

Build order	Avg comparisons per search	10,000 searches
Sorted order	about 3,036	362.73 ms
File order	about 19	2.98 ms
Random shuffle	about 16	2.69 ms
Balanced	about 12	2.05 ms

The sorted chain needs about 260 times more comparisons than the balanced tree and was about 177 times slower. It is not exactly proportional because each search also has a fixed cost (the function call and the loop setup, and copying the code string from `getiata()`), but the order of the rows is exactly what $O(h)$ predicts.

8 Recursive vs. Iterative Versions

To show that the two versions of each operation behave the same, `main()` builds two trees from the file in the same order, one with the iterative `insert` and one with `insertrecursive`. It then runs the same searches and the same five deletes on both, using the iterative functions on the first tree and the recursive ones on the second, and compares the results.

Test	Iterative	Recursive
Build in file order	height 177	height 177, same level order
`find BOS`	12 comparisons	12 comparisons, same path
`find XYZ`	not found, 14 comparisons	not found, 14 comparisons
Minimum / maximum	AAA / ZZV	AAA / ZZV
Delete AAC, GJT, AAA, AEY, GKA	same case printed for each	same case printed for each
Tree after the 5 deletes	same level order (`true`)	

Table 6: Recursive and iterative versions give identical results.

These five deletes run one after another on the same tree (not on a fresh tree each time as in Task 3), so the case printed for AAA is D1: AAC, its only child, was deleted first, which leaves AAA as a leaf. Both versions print the same case at every step and end with the same tree.

Why the program uses the iterative versions. The recursive versions are shorter and match the recursive definition of a BST, but each call uses a stack frame. On the sorted-order tree, a recursive insert or search would go 6,071 calls deep, which is the stack-overflow risk the handout warns about. The iterative versions use a constant amount of extra memory no matter how tall the tree is. The recursive delete is the more elegant of the two delete versions, because it does not need to track a *parent* pointer.

9 Interactive Session

After the tests, the program opens a session with a `>` prompt and the commands named in the handout:

Command	What it does
`find CODE`	search; prints the path, comparisons and the airport
`min`, `max`	smallest / largest code
`remove CODE`	delete; prints the case, the top 3 levels before and after, and validates
`add CODE`	add an airport (asks for name, city, country)
`list`	every airport in code order, with the sorted check
`range FROM TO`	every airport from FROM to TO, and the nodes visited
`save FILE`, `load FILE`	save the codes in preorder / rebuild the tree from that file
`validate`	check the BST property with the range method
`levels`, `stats`	top 3 levels / size, height and average depth
`rebalance`	rebuild as a balanced tree
`help`, `quit`	list the commands / exit

Sample session (also in our demo recording):

```
> find BOS
Path: GKA -> AEY -> EGS -> BUJ -> BJA -> BSK -> BOY -> BNI -> BOH -> BOO -> BOU -> BOS (
    comparisons: 12)
BOS General Edward Lawrence Logan International Airport, Boston, United States
> range BOS BWI
BOS | General Edward Lawrence Logan International Airport | Boston, United States
...
BWI | Baltimore/Washington International Thurgood Marshall Airport | Baltimore, United States
range BOS BWI: 113 airports, 127 nodes visited
> remove AEY
before:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
Deleted AEY (case D3: two children, replaced by successor AFA) validate: OK
after:
  level 0: GKA
  level 1: AFA MAG
  level 2: AAE EGS HGU POM
> validate
validate: true
```

10 Checklist Against the Handout

Requirement	Where it is met	Done
Two classes, `Airport` and `AirportBST`, root private	`src/main.cpp`; `root` and `Node` are private	✓
Loops of the form `for (int i = 0; i < n; i++)`	used for every loop over a vector	✓
Pseudocode before every task	this report and `docs/pseudocode.md`	✓
1a Read the file, handle commas in quotes, skip `\N`	`splitline`, `loadairports`; 6,072 loaded	✓
1b Insert in file order, count, report duplicates	build loop in `main`; 6,072 inserted, 0 duplicates	✓
1c Height and average depth	177 and 18.32	✓
2a `find` prints airport, path, comparisons	`find`, `findrecursive`	✓
2b Root, leaf, deep, missing, smaller, larger, min/max	Task 2 table	✓
3a `remove` with the three cases	`remove`, `removerecursive`	✓
3b D1, D2 left, D2 right, D3, root, with top 3 levels	Task 3 table, `printlevels(3)`	✓
3c Add QQQ, find it, remove it	Section 4.3	✓
4a `list` in order, check sorted	`printinorder`, `isvalidBST`; true	✓
4b `save`/`load` in preorder, same shape	height 177 both, same level order	✓
4c `validate` (range method) after every insert and delete	build loop and `remove`; always true	✓
5 `range` with pruning, nodes visited, 113	113 airports, 127 nodes visited	✓
6 File, sorted, balanced build, timing, explanation	Task 6 table and Section 7.1	✓
6 `rebalance` command	`rebalance()`; height 12	✓
6 Iterative insert and search	`insertBSTiterative`, `searchhelper`, `contains`	✓
6 Random shuffle (bonus)	height 31, average depth 14.88	✓
Public repo `airport-bst-cpp` with `src/`, `data/`, `docs/`	GitHub repository	✓
`docs/pseudocode.md`	build, insert, search, delete, validate, range, balanced build	✓
README: problem, build/run, sample session, Task 6 table, credit	`README.md`	✓
Screen recording or GIF of a session	find, range, remove of a two-child node, validate	✓

11 Problems We Ran Into

- **Integer division.** `averagedepth` first returned `sum / count` with two `int`s, which printed 18 instead of 18.32. Casting to `double` before dividing fixed it.
- **Recursive insert.** We started with a recursive insert. Because the sorted build is a 6,071-node chain, we switched `insert()` to `insertBSTiterative` and kept the recursive version as `insertrecursive`. We then added recursive versions of search, min/max and delete and checked that both versions agree (Section 8).
- **Writing `\N` in C++.** The file uses `\N` for missing values, which has to be written as `"\\N"` in C++, because a single backslash starts an escape sequence.
- **Commas inside names.** Splitting at every comma breaks lines whose names contain commas, so `splitline` tracks whether it is inside quotes.
- **File path.** The program first only opened `../data/airports.dat`, which works when run from `src/` but not from the repository root, where the handout's build command is run. It now tries `data/airports.dat` first, then `../data/airports.dat`.
- **Testing "smaller than every key".** Our first test searched for AAA, but AAA is a real airport (the minimum), so it was found. We changed the test to `000`, which sorts before every letter code.

12 Conclusion

All the milestone checkpoints are met: 6,072 airports are loaded with 0 duplicates, the inorder list is sorted, every search and delete case works with `validate` true after each change, the reloaded tree has the same shape, `range BOS BWI` returns 113, and the Task 6 table is filled in and explained. Every core operation exists in both a recursive and an iterative version, and the two give identical results.

The main thing we learned is that a BST is only as good as its height. The same 6,072 airports gave a height of 12 or 6,071 depending only on the insertion order, and real data files are often partly sorted, which is close to the worst case: our file-order tree was 14 times taller than the balanced one because of a single run of Canadian codes. Rebalancing, or a self-balancing tree such as an AVL or red-black tree, fixes this.

Data Credit

Airport data from **OpenFlights.org** (`https://openflights.org/data.php`), published under the Open Database License (ODbL).

A How to Build and Run

From the repository root:

```
g++ -std=c++17 -O2 src/*.cpp -o airports
./airports (Git Bash, macOS, Linux)
.\airports.exe (Windows PowerShell)
```

The program runs the tests for every task, then opens the interactive session. Repository: `https://github.com/roh9031/airport-bst-cpp`.